

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 7

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 7

TITLE:

Design and develop a Smart Irrigation and Water Management System.
The system automates
irrigation control based on environmental conditions to optimize water
usage in agricultural
applications

Done By:

GURTHARSAN S	- 24MER011
HARISH KUMAR K K	– 24MER015
MADHANRAJ K	– 24MER028
SADAAGOPAAN T R	– 24MER052

Project Overview:

The Smart Irrigation and Water Management System is designed to automatically control irrigation based on environmental conditions and soil moisture. The system helps reduce unnecessary water usage while maintaining suitable moisture levels for crops.

The system uses a soil moisture sensor to determine the moisture content of the soil. An ESP8266 NodeMCU acts as the main controller and processes the sensor data. Based on the soil moisture level, the controller automatically switches the water pump ON or OFF through a relay.

The system can also use temperature and humidity information from a DHT11 sensor to improve irrigation decisions. Since the ESP8266 has Wi-Fi capability, the system can be expanded for remote monitoring through an IoT platform.

Problem Statement

Traditional irrigation systems often supply water according to fixed schedules rather than actual soil conditions. This can result in excessive water usage, insufficient irrigation, and unnecessary pumping.

The major problems are:

- Excessive water consumption.
- Manual irrigation operation.
- Over-irrigation of crops.
- Under-irrigation during dry conditions.
- Unnecessary pump operation.
- Lack of real-time soil moisture monitoring.
- Difficulty in monitoring agricultural fields remotely.

Therefore, an automated irrigation system is required to monitor soil conditions and supply water only when necessary.

Proposed Solution

The proposed system uses an **ESP8266 NodeMCU**, soil moisture sensor, DHT11 sensor, relay module, and water pump.

The soil moisture sensor measures the moisture level of the soil and sends the reading to the ESP8266.

The controller compares the measured value with predefined thresholds.

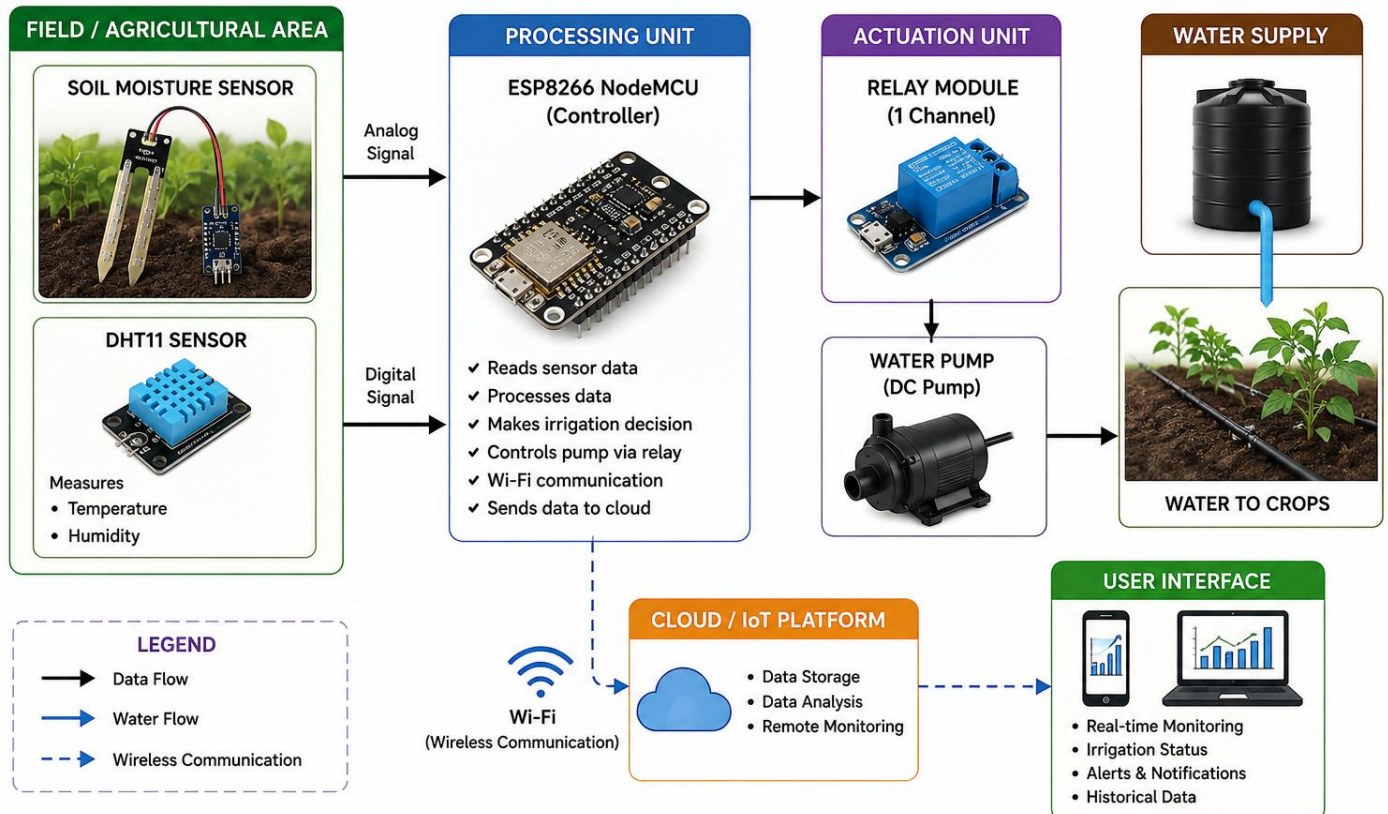
Example Control Logic

Soil Condition	Soil Moisture	Pump
Wet	> 60%	OFF
Moderate	30–60%	OFF
Dry	< 30%	ON

When the soil becomes dry, the ESP8266 activates the relay, which turns ON the water pump. Once sufficient moisture is detected, the pump is switched OFF.

System Architecture

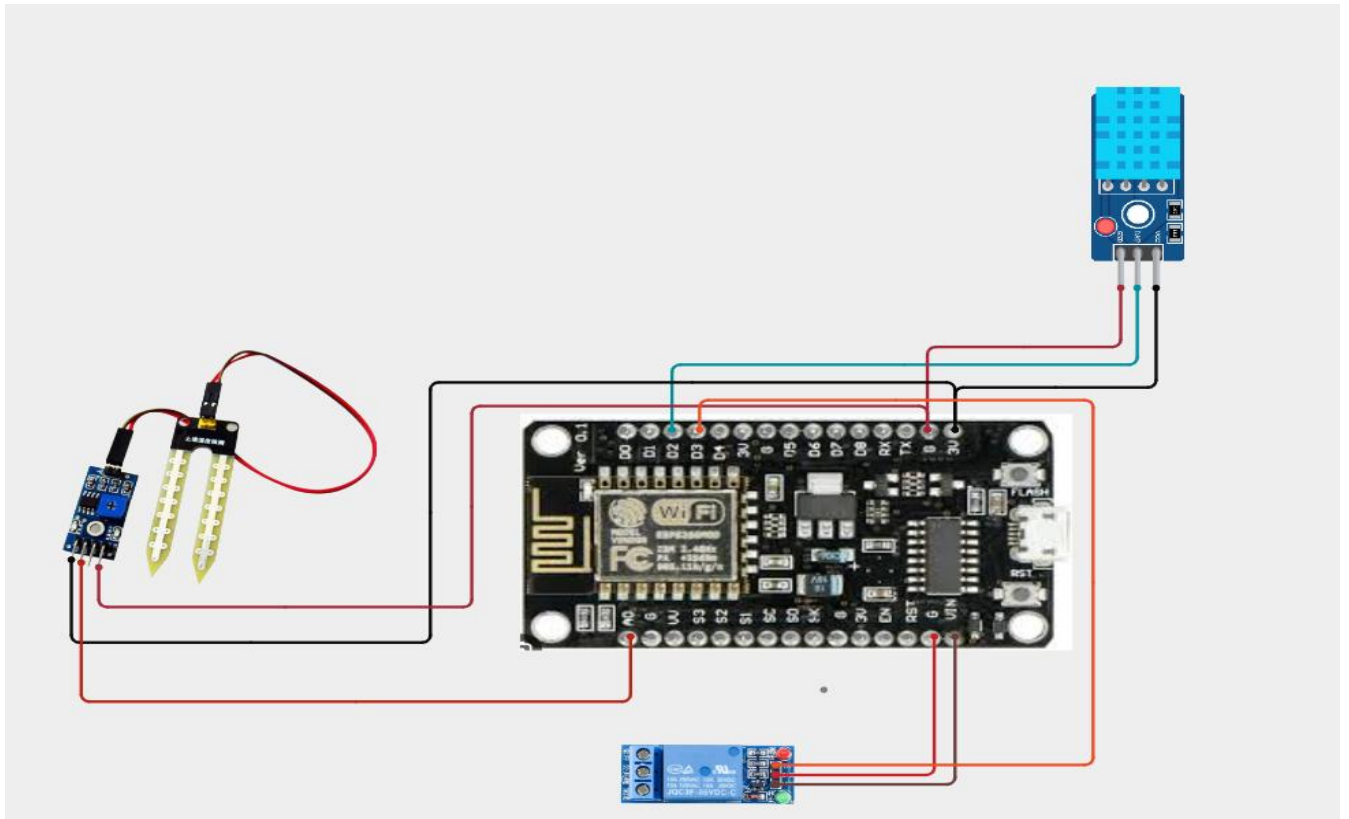
SYSTEM ARCHITECTURE Smart Irrigation and Water Management System



Circuit Table

S.No.	Component	Pin/Terminal	ESP8266 Connection
1	Soil Moisture Sensor	VCC	3.3V
2	Soil Moisture Sensor	GND	GND
3	Soil Moisture Sensor	AO	A0
4	DHT11	VCC	3.3V
5	DHT11	GND	GND
6	DHT11	DATA	D4 (GPIO2)
7	Relay Module	VCC	Suitable supply as specified by module
8	Relay Module	GND	GND
9	Relay Module	IN	D1 (GPIO5)
10	Green LED	Anode	D2 through 220 Ω resistor
11	Green LED	Cathode	GND
12	Yellow LED	Anode	D5 through 220 Ω resistor
13	Yellow LED	Cathode	GND
14	Red LED	Anode	D6 through 220 Ω resistor
15	Red LED	Cathode	GND

Circuit Design



Algorithm

1. START
2. Initialize ESP8266 NodeMCU.
3. Initialize Soil Moisture Sensor.
4. Initialize DHT11 Temperature and Humidity Sensor.
5. Initialize Relay Module and LEDs.
6. Read soil moisture value.
7. Read temperature and humidity.
8. Convert the soil sensor reading into a moisture percentage after calibration.

9. Compare the soil moisture value with the predefined threshold.
10. If soil moisture is below the dry threshold:
 - Turn ON the relay.
 - Turn ON the water pump.
 - Turn ON the irrigation indicator.
 - Display "SOIL DRY - IRRIGATION ON".
11. Else:
 - Turn OFF the relay.
 - Turn OFF the water pump.
 - Turn ON the normal indicator.
 - Display "SOIL MOISTURE SUFFICIENT - IRRIGATION OFF".
12. Display temperature, humidity and soil moisture.
13. Send the sensor data to the IoT dashboard if wireless monitoring is enabled.
14. Wait for a predefined time interval.
15. Read the sensor values again.
16. Repeat the process continuously.
17. STOP

Coding

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>

//===== WiFi Credentials =====
#define WIFI_SSID "Harish Kumar K K"
#define WIFI_PASS "Mavboiii@10"

//===== Adafruit IO =====
#define IO_USERNAME "Jr_Frost"
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

//===== Adafruit IO Feeds =====
AdafruitIO_Feed *soilFeed = io.feed("soil-moisture");
AdafruitIO_Feed *percentFeed = io.feed("moisture-percent");
AdafruitIO_Feed *tempFeed = io.feed("temperature");
AdafruitIO_Feed *humFeed = io.feed("humidity");
AdafruitIO_Feed *pumpFeed = io.feed("pump-status");

//===== GPIO Pins =====
#define SOIL_PIN A0
#define DHTPIN 2    // GPIO2 = D4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

//===== Soil Sensor Calibration =====
// Change these values after testing your sensor
const int DRY_VALUE = 850; // Sensor reading in air
const int WET_VALUE = 300; // Sensor reading in water

// Moisture percentage threshold
const int MOISTURE_THRESHOLD = 40;

void setup() {
  Serial.begin(115200);
```

```

delay(1000);

dht.begin();

Serial.println("\nConnecting to Adafruit IO...");
io.connect();

while (io.status() < AIO_CONNECTED) {
  Serial.print(".");
  delay(500);
}

Serial.println("\nConnected to Adafruit IO!");
}

void loop() {
  io.run();

  int soilRaw = analogRead(SOIL_PIN);

  // Convert raw value to percentage
  int moisturePercent = map(soilRaw, DRY_VALUE, WET_VALUE, 0, 100);
  moisturePercent = constrain(moisturePercent, 0, 100);

  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  // Upload data
  soilFeed->save(soilRaw);
  percentFeed->save(moisturePercent);
  tempFeed->save(temp);
  humFeed->save(hum);

  Serial.println("-----");
  Serial.print("Soil Raw Value : ");
  Serial.println(soilRaw);

  Serial.print("Moisture      : ");
  Serial.print(moisturePercent);
  Serial.println("%");

```

```

Serial.print("Temperature   : ");
Serial.print(temp);
Serial.println(" °C");

Serial.print("Humidity     : ");
Serial.print(hum);
Serial.println("%");

if (moisturePercent < MOISTURE_THRESHOLD) {
  Serial.println("Water Pump ON - Soil is Dry");
  pumpFeed->save(String("ON"));
} else {
  Serial.println("Water Pump OFF - Soil is Wet");
  pumpFeed->save(String("OFF"));
}

if (temp > 30.0) {
  Serial.println("High Temperature Detected!");
}

delay(5000);
}

```

System Working

The system starts by initializing the ESP8266, soil moisture sensor, DHT11 sensor, relay, LEDs, and water pump control.

The soil moisture sensor continuously measures the moisture condition of the soil.

Dry Soil

When the soil moisture falls below the predefined threshold:

Relay → ON

Water Pump → ON

Water is supplied to the crop.

Moist Soil

When the moisture reaches a sufficient level:

Relay → OFF

Water Pump → OFF

This prevents unnecessary water usage.

The DHT11 simultaneously measures temperature and humidity. These values can be displayed locally and transmitted through Wi-Fi if IoT functionality is added.

Bill of Materials

S.N o.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU	Wi-Fi Microcontroller	1	Main Controller
2	Soil Moisture Sensor	Capacitive Type	1	Soil Moisture Detection
3	DHT11 Sensor	Temperature & Humidity	1	Environment al Monitoring
4	Relay Module	1-Channel	1	Pump Switching
5	DC Water Pump	Suitable Low- Voltage Pump	1	Water Supply
6	Green LED	5 mm	1	Normal/Mois t Condition
7	Yellow LED	5 mm	1	Monitoring Indication
8	Red LED	5 mm	1	Irrigation/Dry Condition
9	Resistor	220 Ω	3	LED Current Limiting
10	Breadboard	Standard	1	Circuit Assembly
11	Jumper Wires	Male-Male	1 Set	Connections
12	Power Supply	Suitable for ESP8266 & Pump	1	Power Supply

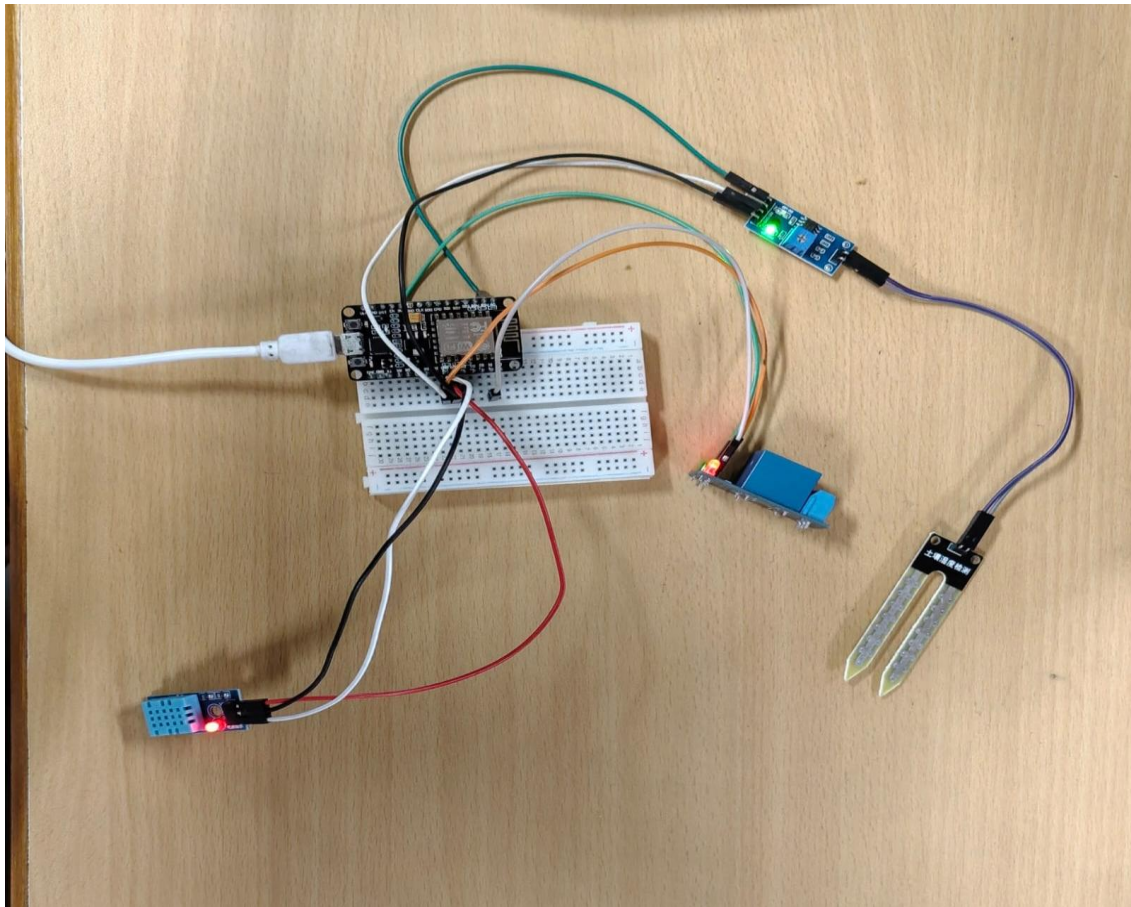
Prototype Implementation

The prototype is assembled using an ESP8266 NodeMCU, capacitive soil moisture sensor, DHT11, relay module, water pump, LEDs, breadboard, and jumper wires.

The soil moisture sensor is inserted into the soil near the crop root zone. The DHT11 sensor is placed above or near the crop area to measure environmental temperature and humidity.

The ESP8266 continuously reads the sensor values.

When the soil becomes dry, the controller activates the relay and the pump supplies water to the soil. When adequate moisture is restored, the pump automatically switches OFF.



Results and Performance Analysis

The system successfully demonstrates automatic irrigation based on soil moisture.

Parameter	Result
Soil Moisture Monitoring	Yes
Temperature Monitoring	Yes
Humidity Monitoring	Yes
Automatic Pump Control	Yes
Wireless Connectivity	Yes
Manual Irrigation Requirement	Reduced
Water Management	Improved
IoT Expansion	Supported

Performance Analysis

The system reduces unnecessary irrigation by operating the pump according to actual soil moisture rather than a fixed schedule.

The ESP8266 provides wireless connectivity, allowing the system to be expanded for remote monitoring.

The effectiveness of the system depends strongly on soil-moisture sensor calibration, sensor placement, crop type, soil characteristics, irrigation method, and threshold selection.

Soil Moisture: 1024
Temperature: 28.30 °C
Humidity: 70.80 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.30 °C
Humidity: 70.80 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.30 °C
Humidity: 70.80 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.30 °C
Humidity: 70.80 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.20 °C
Humidity: 71.00 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.20 °C
Humidity: 71.00 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.20 °C
Humidity: 71.50 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.30 °C
Humidity: 71.40 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.20 °C
Humidity: 71.40 %
Water Pump ON - Soil is Dry

Soil Moisture: 1024
Temperature: 28.20 °C
Humidity: 71.50 %
Water Pump ON - Soil is Dry

